

Практическая работа №7

Использование технологий PHP для обработки данных на форме

Цель работы: получить практические навыки обработки данных на форме с помощью технологий PHP.

Лабораторное оборудование: персональный компьютер с доступом в Интернет.

Время: 2 часа.

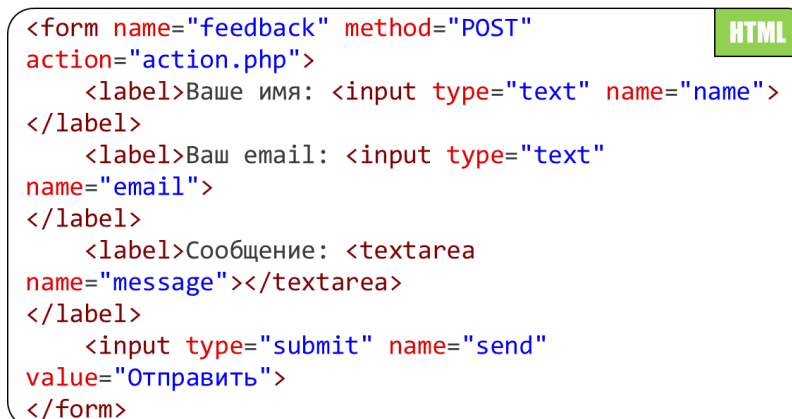
Краткие теоретические сведения

Формы – это часть языка HTML. Формы нужны для передачи данных от клиента на сервер. Чаще всего формы используются для регистрации пользователей, заполнения анкет, оформления заказа в интернет-магазине и т.д.

Через формы можно отправлять как простую текстовую информацию, так и файлы. Изучение языка программирования PHP, так или иначе затрагивает работу с формами и данными, которая эта форма передаёт.

Форма с точки зрения HTML

Описывает то, из каких элементов состоит форма и как она выглядит. Но без принимающей стороны, т.е. сервера, который принимает эти данные и обрабатывает их нужным образом, создавать формы нет никакого смысла (рисунок 7.1).



```
<form name="feedback" method="POST"
action="action.php">
  <label>Ваше имя: <input type="text" name="name">
</label>
  <label>Ваш email: <input type="text"
name="email">
</label>
  <label>Сообщение: <textarea
name="message"></textarea>
</label>
  <input type="submit" name="send"
value="Отправить">
</form>
```

Рисунок 7.1 – Пример формы с точки зрения HTML

Формы в HTML используются для сбора данных от пользователя. Данные, отправленные через форму, могут быть обработаны на сервере с помощью PHP. Основные элементы формы:

- **<form>** — контейнер для формы.

- `<input>` — поле ввода (текст, пароль, чекбокс, радиокнопка и т.д.).
- `<textarea>` — многострочное текстовое поле.
- `<select>` — выпадающий список.
- `<button>` — кнопка для отправки формы (также можно использовать `input` с соответствующим типом).

Форма с точки зрения PHP

Содержит множество средств для работы с формами. Это позволяет очень просто решать типичные задачи, которые часто возникают в веб-программировании (рисунок 7.2):

- Регистрация и аутентификация юзера;
- Отправка комментариев в соц. сетях;
- Оформление заказов товаров или заказ справки об обучении...

```
<?php
if (isset($_POST)) {
    echo "Имя: " . $_POST['name'];
    echo "Email: " . $_POST['email'];
    echo "Сообщение: " . $_POST['message'];
} ?>
```

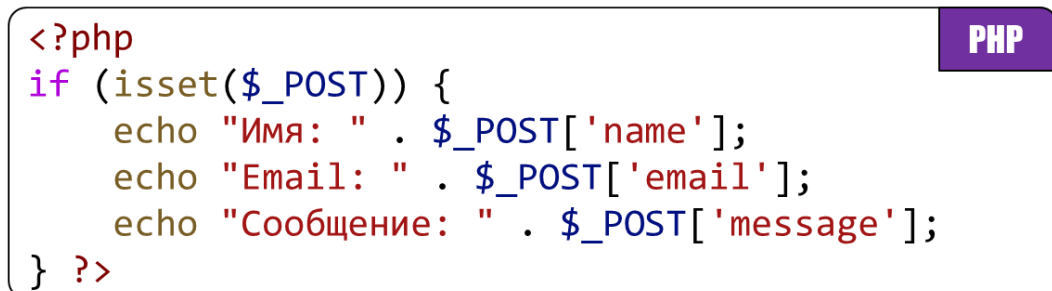


Рисунок 7.2 – Пример обработки формы с точки зрения PHP

Методы отправки данных

GET. Данные передаются через URL. Подходит для небольших объёмов данных:

```
<form method="GET" action="action.php">
```

POST. Данные передаются в теле HTTP-запроса. Подходит для больших объёмов данных и конфиденциальной информации.

```
<form method="POST" action="action.php">
```

Обработка данных в PHP

Данные из формы доступны в PHP через суперглобальные массивы:

- `$_GET` — для данных, отправленных методом GET.
- `$_POST` — для данных, отправленных методом POST.

Пример обработки данных:

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $name = $_POST['name'];  
    $email = $_POST['email'];  
    echo "Имя: $name, Email: $email";  
}
```

Задания на выполнение практической работы

Примечание. Для выполнения поставленной задачи рекомендуется использовать специальное программное обеспечение **Visual Studio Code** (VS Code) или аналог в виде **Sublime Text** (ST). Для удобства разработки рекомендуется установить расширение **Live Server** by Ritwick Dey (идентификатор расширения: ritwickdey.LiveServer).

Также для выполнения поставленной задачи вам понадобится Open Server, он уже установлен на рабочих компьютерах, поэтому никаких действий по установке не требуется.

Задание 1. Найдите в поиске приложение «Open Server Panel» и запустите его (рисунок 7.3).

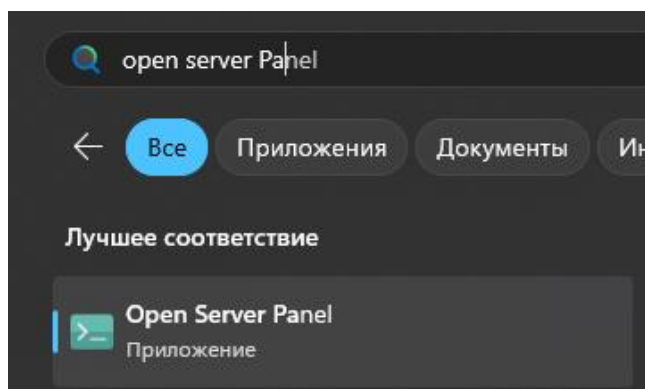


Рисунок 7.3 – Поиск и запуск приложения «Open Server Panel»

Задание 2. Если всё выполнено корректно, в трее вы увидите зеленый флажок – это и есть наш локальный сервер (Open Server Panel). Нажмите на него правой кнопкой мыши (ПКМ) и откройте папку с проектами (рисунок 7.4).

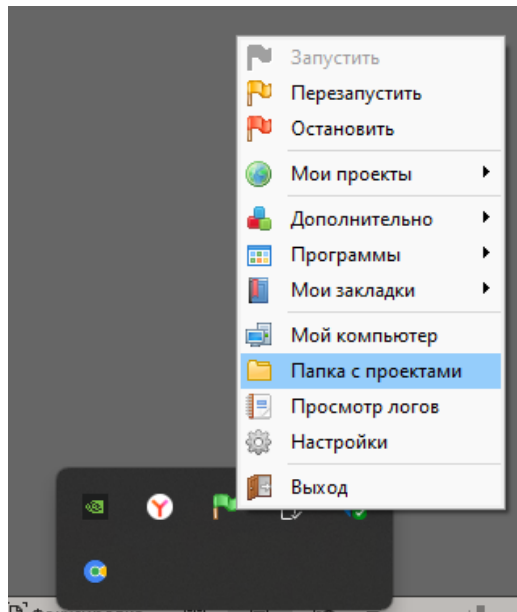


Рисунок 7.4 – Запущенный локальный сервер «Open Server Panel»

Задание 3. После открытия папки с проектами в проводнике Windows, создайте там папку, назовите её как-нибудь на английской раскладке, без пробелом. Например: *MySite*.

Задание 4. Скопируйте путь к данной папке в адресной строке проводника windows как показано на рисунке 7.5.

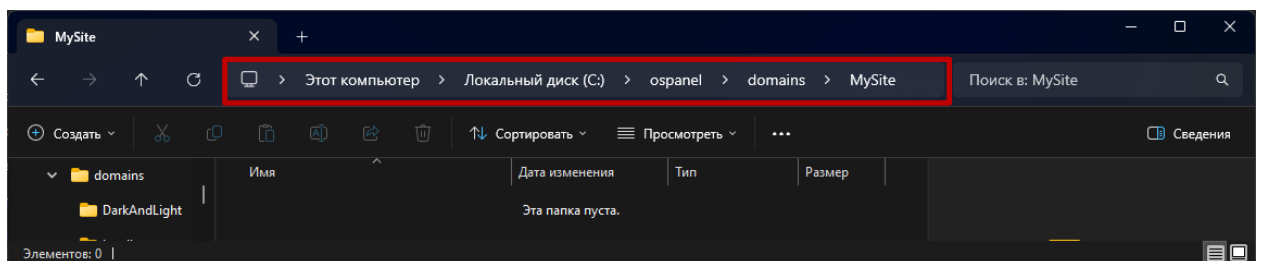


Рисунок 7.5 – Скопируйте путь к папке в проводнике Windows

Задание 5. Запустите программное обеспечение Visual Studio Code и выполните следующую команду: *файл → открыть папку (Ctrl + K + O)*. В открывшемся окне проводника Windows, вставьте в адресную строку путь, который скопировали в задании 4.

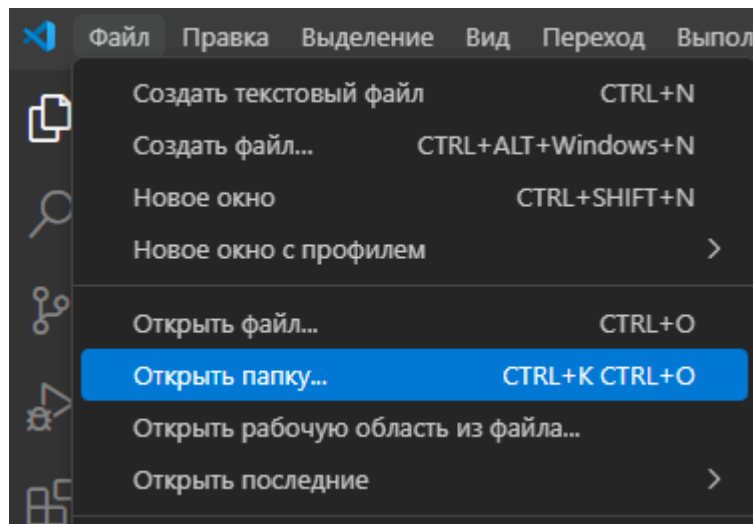


Рисунок 7.6 – Выполнение команды файл → открыть папку

*Отлично, у вас всё получилось! Теперь эта папка – ваш маленький проект, который будет работать на локальном сервере, теперь мы можем работать с файлами расширения *.php и открывать их в браузере.*

Задание 6. Воспользуйтесь вашей формой, которую вы создавали ранее на лабораторной работе. Создайте PHP-скрипт «action.php», который будет обрабатывать данные из формы. Выведите полученные данные на страницу. Добавьте проверку на пустоту для всех полей формы.

Подсказка: для проверки поля на пустоту, воспользуйтесь функцией `empty()`.

Задание 7. Модернизируйте ваш код: добавьте валидацию поля **email** (если такого поля на форме нет, создайте его).

Подсказка: для валидации поля **email** воспользуйтесь функцией `filter_var()`, где следует первым параметром указать входную строку с самим адресом электронной почты, а в качестве второго параметра передать `FILTER_VALIDATE_EMAIL`. В случае успеха, функция вернёт true, если адрес электронной почты будет действителен.

Задание 8. Сделайте проверку на дурака. Если пользователь вводит некорректные данные или пытается отправить форму с пустыми полями, предупредите его, что так делать нельзя. Воспользуйтесь `echo` для вывода сообщения на странице средствами PHP.

Задание 9. Модернизируйте ваш код так, чтобы полученные данные с формы заносятся в текстовый файл, который потом необходимо прочитать и отобразить его содержимое на странице. Имя для текстового файла выберите самостоятельно.

Контрольные вопросы

1. Какие методы отправки данных формы вы знаете? В чём их отличие?
2. Как получить данные из формы в PHP? Приведите пример использования суперглобального массива \$_POST.
3. Для чего нужна валидация данных? Какие функции PHP вы использовали в данной работе для валидации данных?
4. Как проверите входную строку на пустоту? Приведите пример.
5. В каких случаях лучше всего использовать GET или POST запросы для отправки формы?

Содержание отчёта

Отчет выполняется единым, в текстовом (электронном) виде, который содержит номер лабораторной/практической работы, тему и цель работы, а также краткое описание выполняемых операций и скриншоты результатов выполнения, ответы на контрольные вопросы, выводы. Отчёт оформляется согласно ГОСТ 7.32-2017.

Структура отчёта должна содержать следующее:

- наименование и номер работы;
- тему и цель работы;
- раздел «Постановка задачи», где описываются задачи, которые необходимо решить в ходе выполнения практической работы;
- раздел «Ход работы», где описывается ход выполнения работы с кратким описанием выполняемых операций и приведенных результатов выполнения в виде скриншотов;
- листинг программы (при наличии, в зависимости от поставленной задачи);
- ответы на контрольные вопросы;
- выводы (приводятся исходя из полученных знаний и выполненных работ, **указание выводов основываясь только из поставленной цели работы не допускается!**).